

Use main.java as single
launch point

Notes default package has a lot of commented out code in app because that was used for testing my sql connection. AppUI and Main will work the same as the singular run point AppUI has the commented out register window incase we go over something in class and i want to look at it the same way there. PasswordUtil is not my code as my hashes wouldn't work. If you want me to scrap it and try to wright it my self in milestone 3 let me know and i will do so. The sql was confusing on how to establish the connection which is what lead to creation of the databaseConnection class and I'm excited to see how we do it in class because i simply followed a tutorial and it didn't sink in.

P.S. i went to milesstone 2 to do one last read through and saw we didn't need login implemented yet lmao

Clearly the data will not reset when app closes as it has a database connection. If i didnt supply every thing for them in the SQLTable folder please email me what you need and ill get it to you

I suggest using these
username : johndoe

Password : password123

There are a few you can see the password to but any new ones the passwords will be hashed

A) What you implemented (overview)

- one paragraph describing what works in this milestone
I have a full data base and connection for users and bugs. In order to login you need to use a propper pair of username or email with there matched password for that user. New user requires all fields to be full email has to be formatted correctly, password must be 8 long one capital and one number. The passwords must match. When adding a new bug or editing one you most have all fields filled except description. You can delete, refresh to clear search filter results and quick update to in progress or complete. You can filter 4 diffrent ways status priority who made the bug and keywords. Part of why this is late password hashing was just added took longer than expected and broke everything half way through.
ps i now love ctrl + shift + f

B) Features completed (checklist)

Use main.java as single
launch point

- list the features you have working (see requirements below)
 - ✓ a main window (JFrame)
 - ✓ a **JTable** (or equivalent) showing multiple records
 - ✓ buttons or menu actions for:
 - **Refresh/List All**
 - **New/Create**
 - **Edit/Update**
 - **Delete**
 - ✓ a create/edit dialog (JDialog) with:
 - input fields for key attributes
 - OK / Cancel
 - edit mode should populate existing values
 - ✓ A basic data base connection using MYSQL for users and bugs
 - ✓ Fully working register new user and locked out login verifier
 - ✓ Working search filters
 - ✓ Button that switches to in progress and one that changes to complete

C) Architecture summary

Include a short section describing your layering and packages. Your code must follow:

- **UI (View) → Controller → Service → DAO (in-memory)**
- Mine goes like so
- **View -> controller-> service -> dao -> database** with model used by all
i also have a package called util i put my hasher in there
- **Model**
 - This is the base of the app it relies on no other package but all others rely on it
 - Has Bug, User, BugStatus, BugPriority
lots of getter and setters and holds data
- **Database**
 - Has DatabaseConnection
connects to DAO its only job is to handle the connection to MySQL
- **Dao**
 - Talks to database directly
 - Has BugDaoImplementation, UserDaoImplementation, BugInterface, UserDaoInterface
 - These have all the SQL and rely on model for the objects and database for connection

Use main.java as single launch point

- **Service**
 - **Has BugService and UserService**
these take care of login validation, connects to util for hashing and filtering
 - **Connects to dao and model**
- **Controller**
 - **Has LoginController, RegisterController, BugListController, AddUpdateBugController**
 - Reads input from view and calls ether dao or service to then let view know what to show
 - Connects to dao, view, model, service
- **View**
 - **Has LoginView, RegisterView, BugListView, AddUpdateBugView**
 - Connects to controller and model
 - Displays data and has layout of what to show and gets whats fired
- **Util**
 - **Has PasswordUtil**
 - **Has the password hasher connects to user service to hash the password for it**

D) Business rules and validation

- list at least 1–2 rules/validations implemented in the service layer (e.g., required fields, uniqueness, format checks)
- In making a new user email has to have @ something . Something
- Also password must have at least 8 char, one number and a capital letter
- user name cant already exist
- All fields must be filled in register
- The due date on bugs must be after the current day and follow (YYYY-MM-DD)
- All in new bug except description must be filled

E) Known limitations / planned next steps

- a short bullet list (what's missing, what you'll add in Milestone 3)
- Admin specific stuff like users and accounts view add full version of this so delete edit and more are there
- A extra filter catagory for what type of bug it is
- Change due date format to U.S.A standard

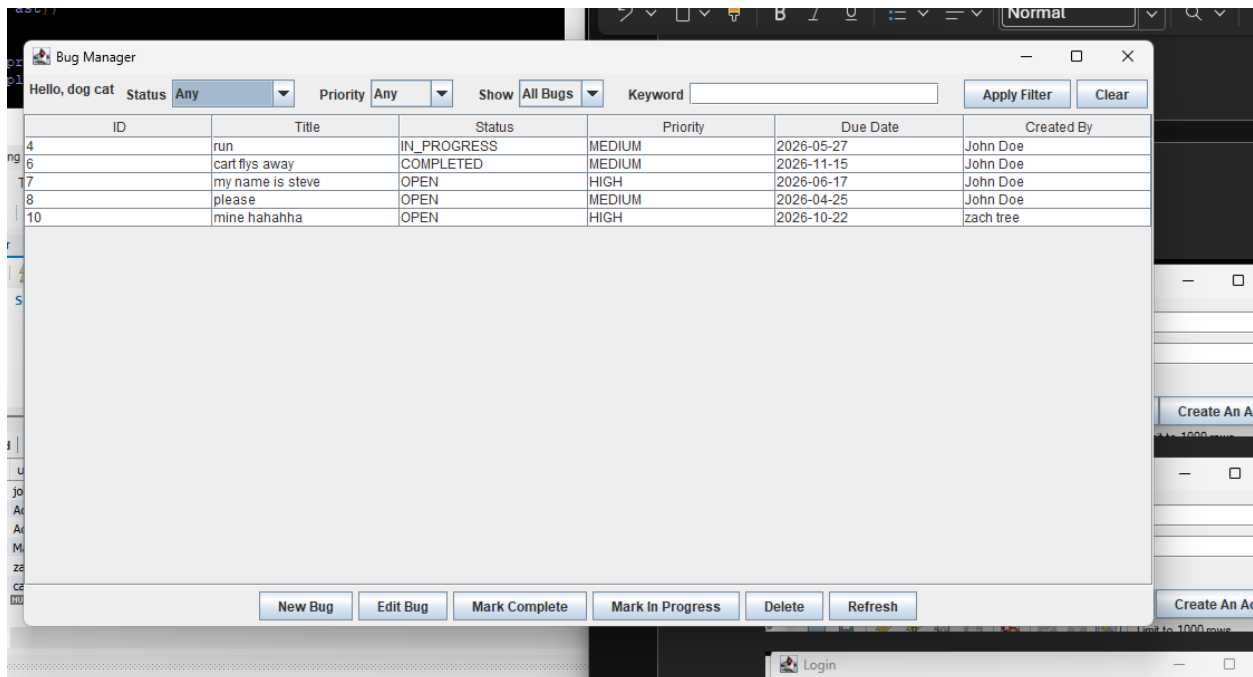
Use main.java as single launch point

- A way to view the bug with out clicking edit
- The ids get incremented higher sometimes when the user gets flagged for not filling everything out properly. Please in the notes let me know if i should fix that also please let me know if i should have it detect the lowest id not in use be for generating the next one. Currently it fetches the id auto generated by MySQL.
- The code needs way more comments. Most comments are from what we did in class together, whoevers tutorial i was watching or example i found to follow.
- I hope to have the time to make a app launch screen
- Also im assuming there should be a way to make someone an admin in app and im wondering how we should empliment that

3) Screenshots

Include screenshots in your PDF/DOCX (or as separate images) showing:

- the main window with the table/list visible



- the create dialog

Use main.java as single launch point

Bug Manager

Hello, dog cat

Status Any Priority Any Show All Bugs Keyword

Apply Filter Clear

ID	Title	Status	Priority	Due Date	Created By
4	run	IN_PROGRESS	MEDIUM	2026-05-27	John Doe
6	cart flies away	COMPLETED	MEDIUM	2026-11-15	John Doe
7	my name is steve	OPEN	HIGH	2026-06-17	John Doe
8	please mine				John Doe
10					zach tree

New Bug

Title

Description

Due Date (YYYY-MM-DD)

Priority MEDIUM

Save

New Bug Edit Bug Mark Complete Mark In Progress Delete Refresh

[illegible]

Use main.java as single launch point

MySQL Workbench

Local instance MySQL96 (db_task.x) Spring 2026 java - V

File Edit View Query Database Server Tools Script

Navigator

SCHEMAS

Filter objects

bug_tracker

bugs

Columns

Indexes

Foreign Keys

Triggers

users

Columns

Indexes

Foreign Keys

Triggers

Views

Stored Procedures

Functions

db_task_manager_project

Tables

tb_task

Administration Schemas

Information

Table: tb_task

Columns:

id int AI PK

user_id int

title varchar(255)

description longtext

due_date date

status varchar(255)

priority varchar(255)

Object Info Session

New Bug

Title money dupe

Description by spamming x fast enough you can sell one item multi times

Due Date (YYYY-MM-DD) 2026-04-05

Priority HIGH

Save

Result Grid

#	id	user_id	title	description	create_date	due_date	status	priority
4	1	run	run run im the gergerbread man you c...		2026-03-27	2026-05-27	IN_P...	MEDIUM
6	1	cart flys...			2026-03-29	2026-11-15	COM...	MEDIUM
7	1	my nam...	aaaaaaaaaaaaahhh		2026-03-29	2026-06-17	OPEN	HIGH
8	1	please	yahsakhf		2026-03-29	2026-04-25	OPEN	MEDIUM
10	8	mine ha...	hide if u can		2026-03-29	2026-10-22	OPEN	HIGH

bugs 1 x

Action Output

#	Time	Action	Message	D
6	15:49:44	SELECT * FROM bug_tracker.users LIMIT 0, 1000	0 row(s) returned	0.
7	15:49:59	SELECT * FROM bug_tracker.bugs LIMIT 0, 1000	0 row(s) returned	0.
8	07:41:51	SELECT * FROM bug_tracker.bugs LIMIT 0, 1000	1 row(s) returned	0.
9	07:41:54	SELECT * FROM bug_tracker.users LIMIT 0, 1000	1 row(s) returned	0.
10	09:10:06	SELECT * FROM db_task_manager_project.tb_task LIMIT 0, 1000	0 row(s) returned	0.
11	09:10:20	ANALYZE TABLE 'db_task_manager_project'.tb_task	OK	0.

MySQL Workbench

Local instance MySQL96 (db_task.x) Spring 2026 java - V

File Edit View Query Database Server Tools Script

Navigator

SCHEMAS

Filter objects

bug_tracker

bugs

Columns

Indexes

Foreign Keys

Triggers

users

Columns

Indexes

Foreign Keys

Triggers

Views

Stored Procedures

Functions

db_task_manager_project

Tables

tb_task

Administration Schemas

Information

Table: tb_task

Columns:

id int AI PK

user_id int

title varchar(255)

description longtext

due_date date

status varchar(255)

priority varchar(255)

Object Info Session

New Bug

Title money dupe

Description by spamming x fast enough you can sell one item multi times

Due Date (YYYY-MM-DD) 2026-04-05

Priority HIGH

Save

Message

Bug saved!

OK

Result Grid

#	id	user_id	title	description	create_date	due_date	status	priority
4	1	run	run run im the gergerbread man you c...		2026-03-27	2026-05-27	IN_P...	MEDIUM
6	1	cart flys...			2026-03-29	2026-11-15	COM...	MEDIUM
7	1	my nam...	aaaaaaaaaaaaahhh		2026-03-29	2026-06-17	OPEN	HIGH
8	1	please	yahsakhf		2026-03-29	2026-04-25	OPEN	MEDIUM
10	8	mine ha...	hide if u can		2026-03-29	2026-10-22	OPEN	HIGH
11	9	money ...	by spamming x fast enough you can sell one ...		2026-03-30	2026-04-05	OPEN	HIGH

bugs 1 x

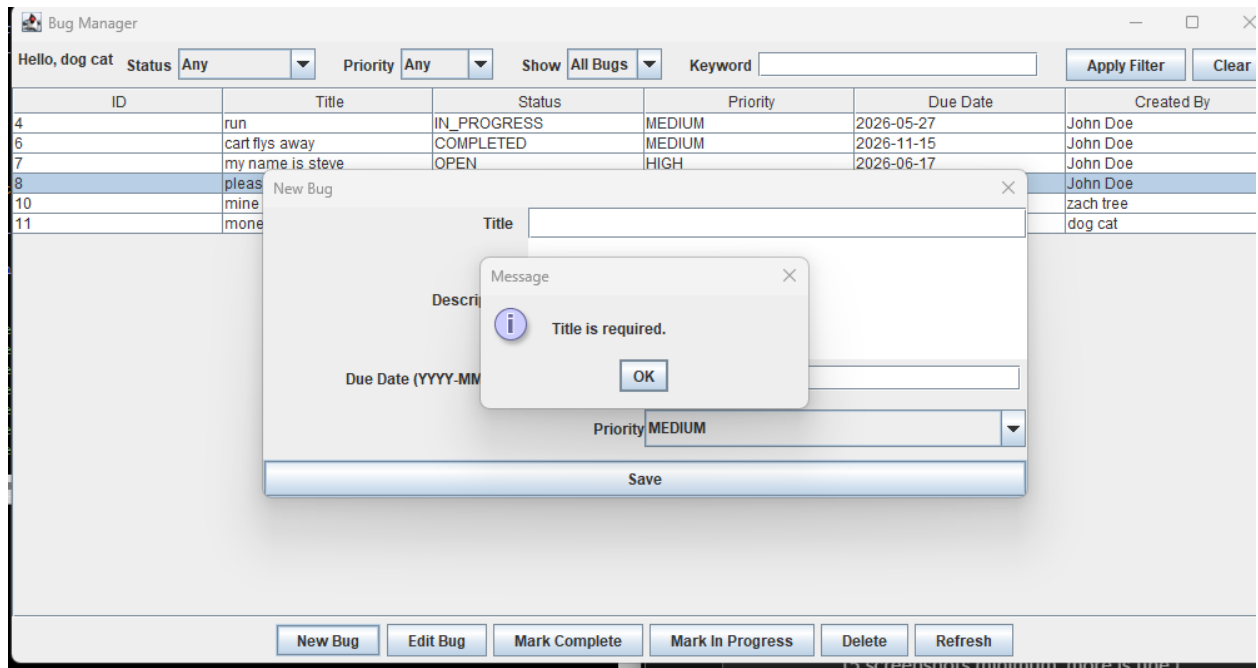
Action Output

#	Time	Action	Message	Duration / Fetch
6	15:49:44	SELECT * FROM bug_tracker.users LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
7	15:49:59	SELECT * FROM bug_tracker.bugs LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
8	07:41:51	SELECT * FROM bug_tracker.bugs LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
9	07:41:54	SELECT * FROM bug_tracker.users LIMIT 0, 1000	1 row(s) returned	0.000 sec / 0.000 sec
10	09:10:06	SELECT * FROM db_task_manager_project.tb_task LIMIT 0, 1000	0 row(s) returned	0.000 sec / 0.000 sec
11	09:10:20	ANALYZE TABLE 'db_task_manager_project'.tb_task	OK	0.000 sec

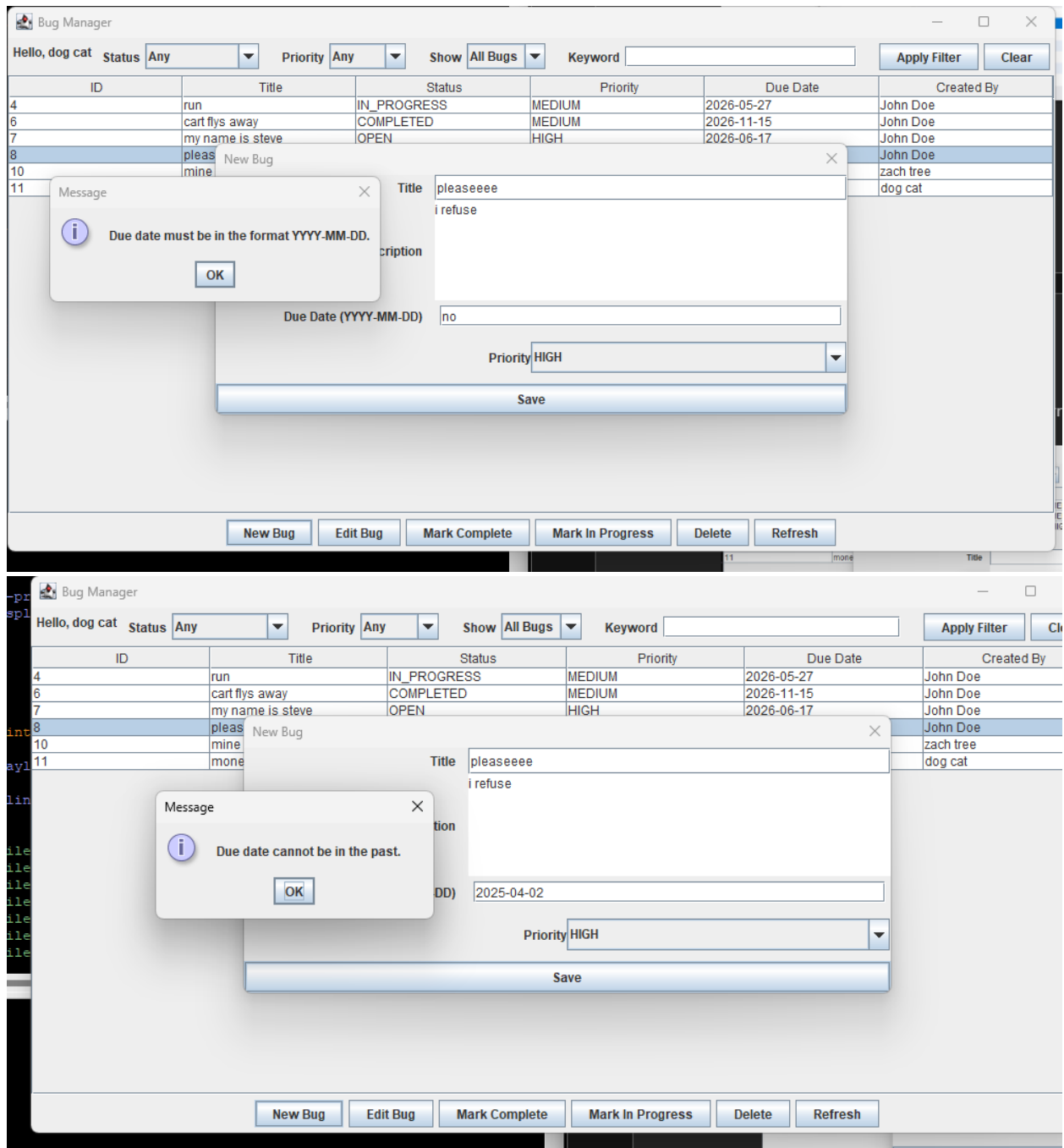
Query Completed

- an example error message dialog (validation error)

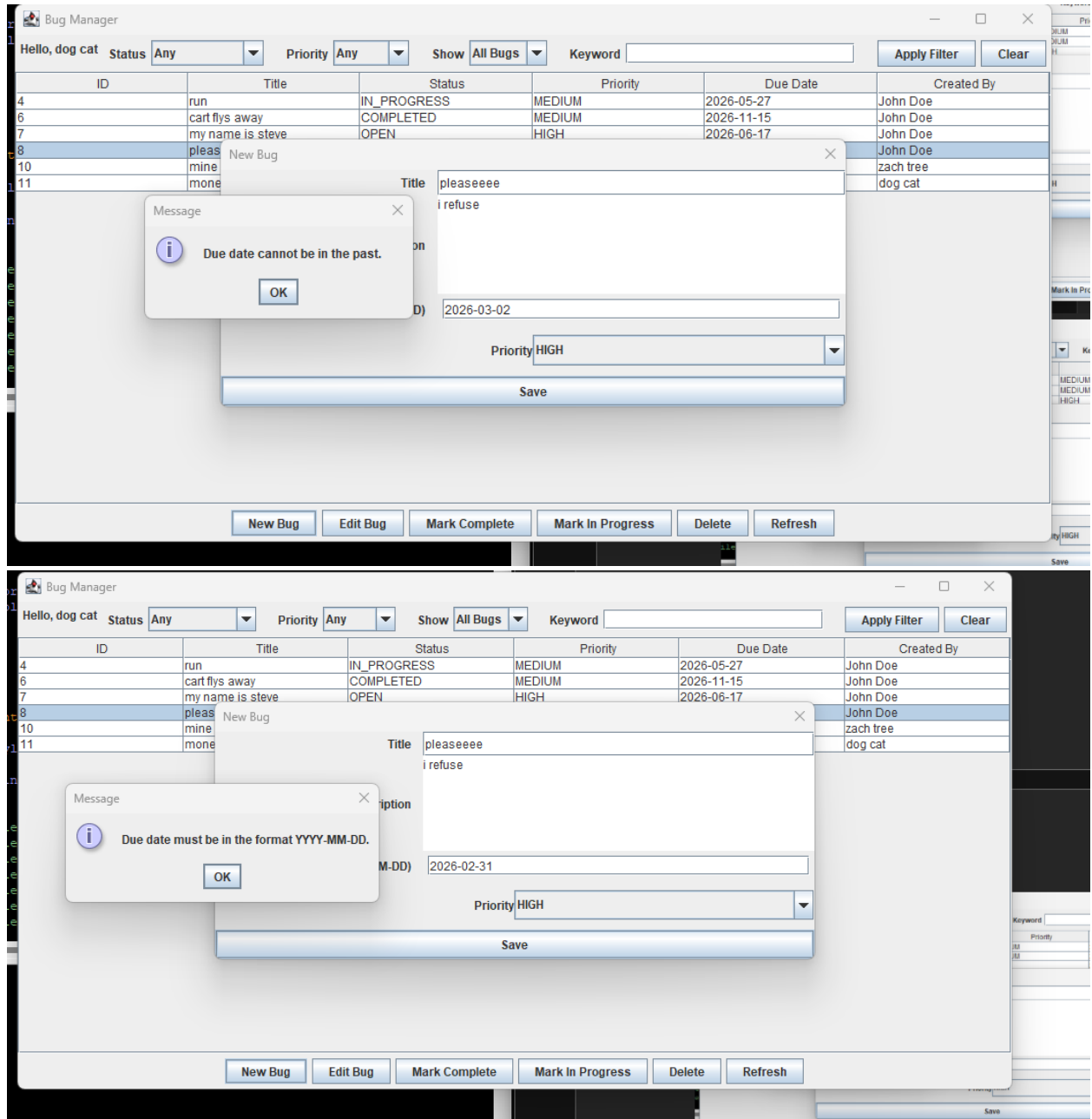
Use main.java as single
launch point



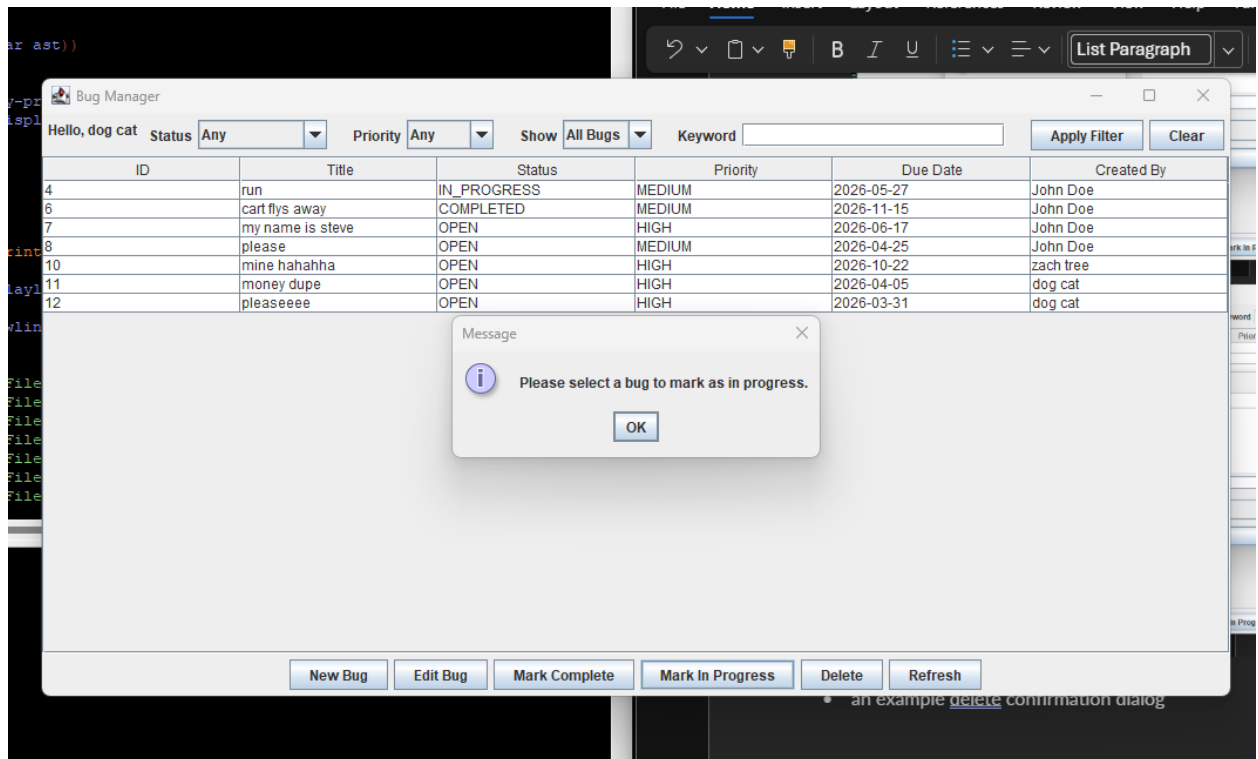
Use main.java as single
launch point



Use main.java as single
launch point

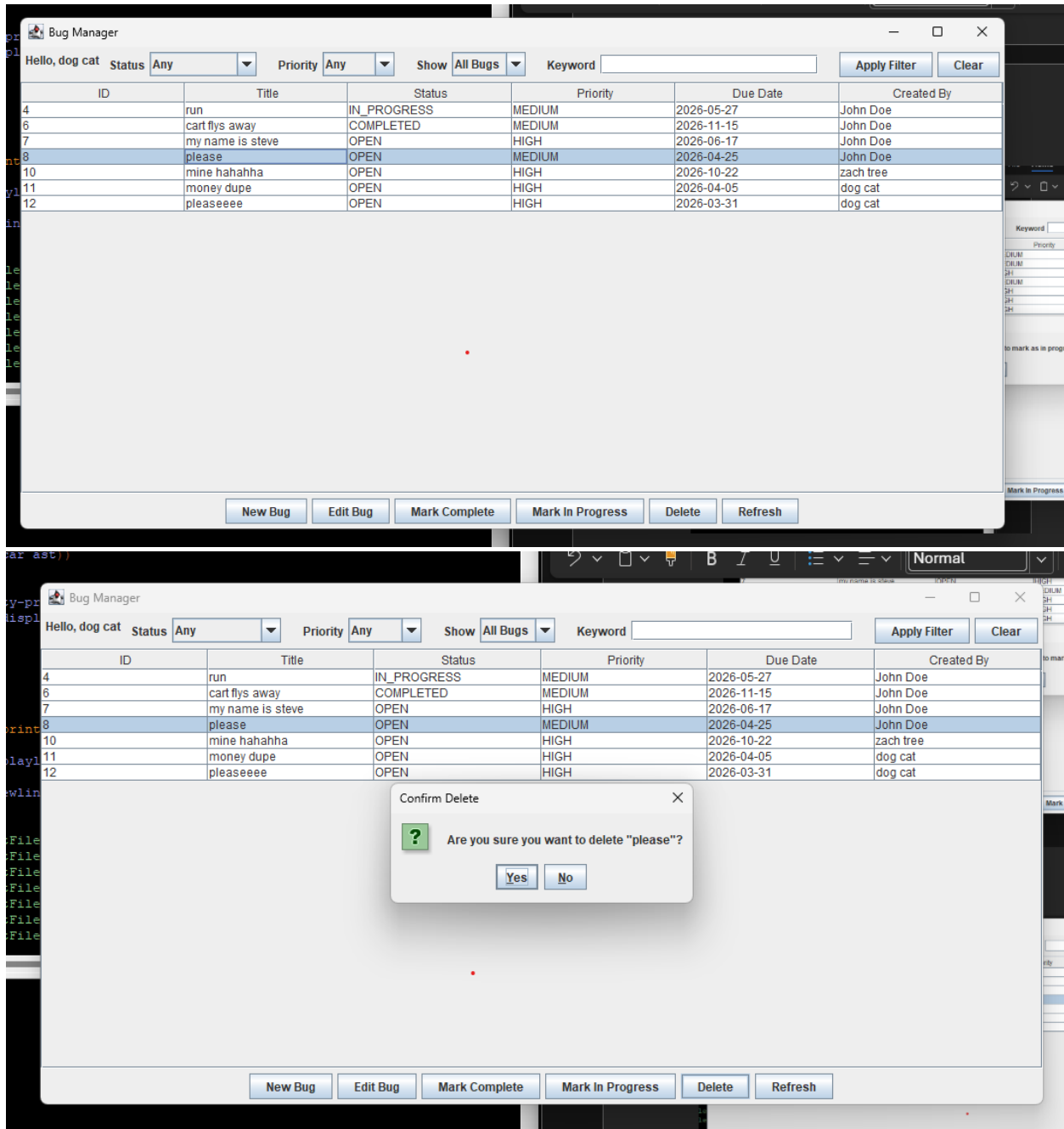


Use main.java as single
launch point

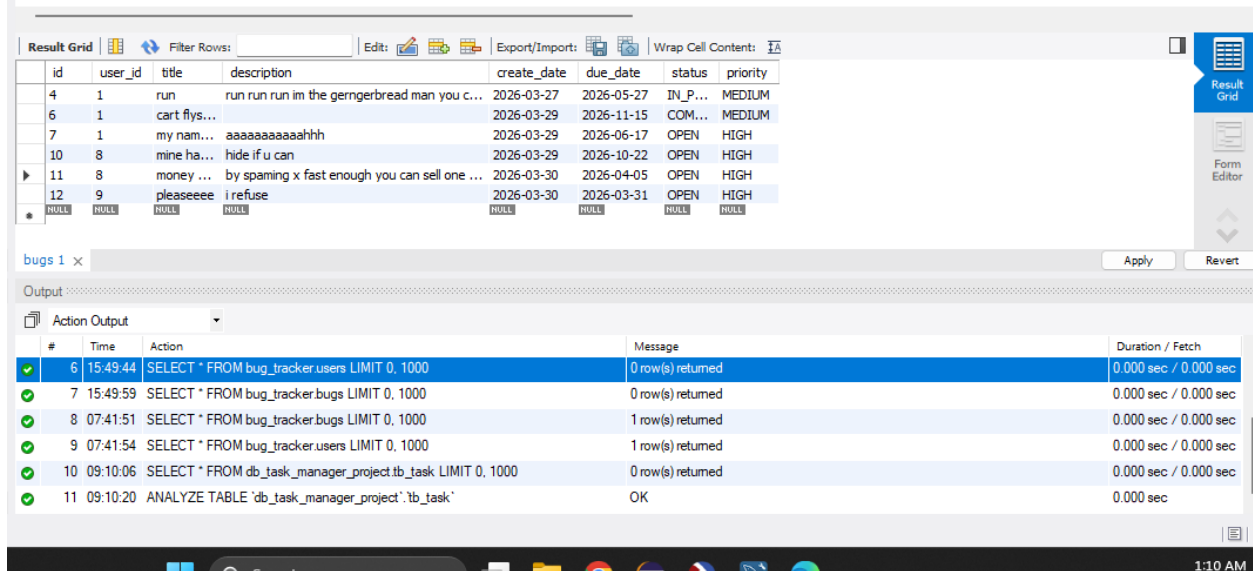
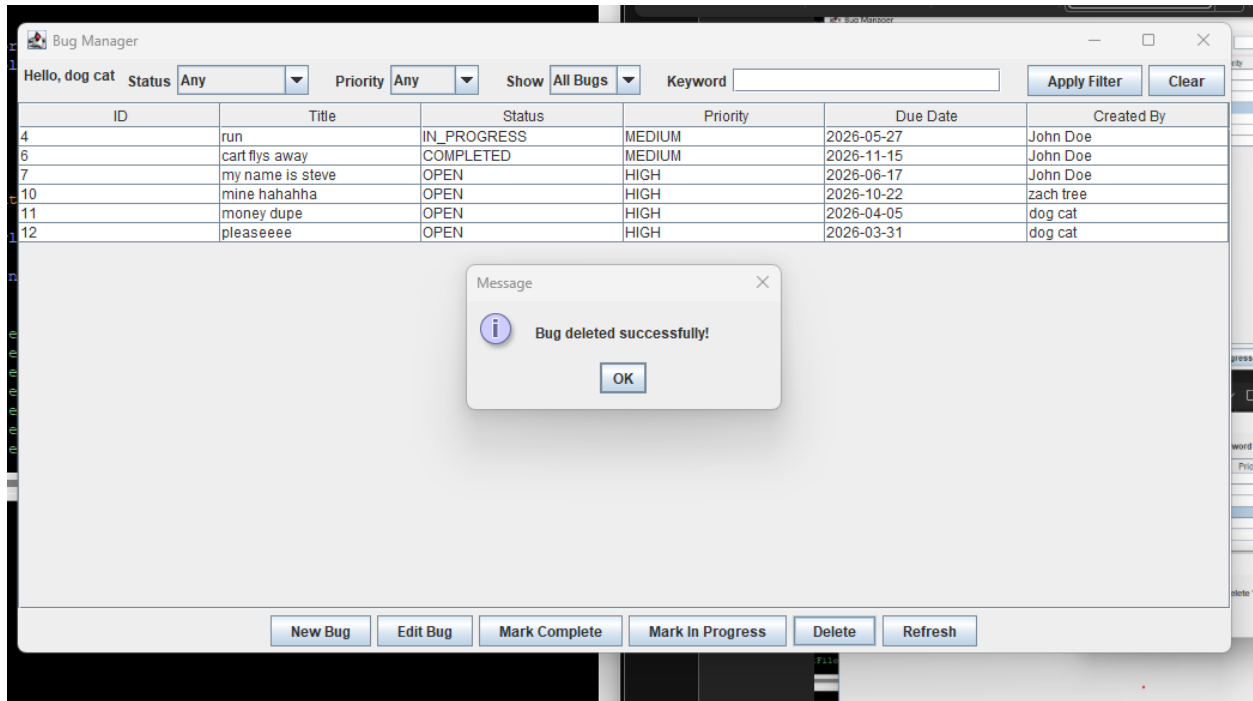


- an example delete confirmation dialog

Use main.java as single
launch point

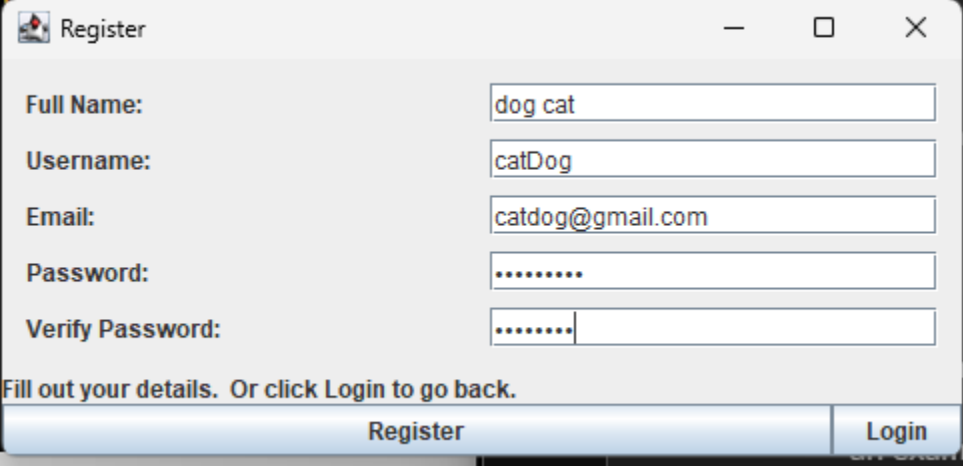


Use main.java as single
launch point



(5 screenshots minimum, more is fine.)

Use main.java as single
launch point



Register

Full Name: dog cat

Username: catDog

Email: catdog@gmail.com

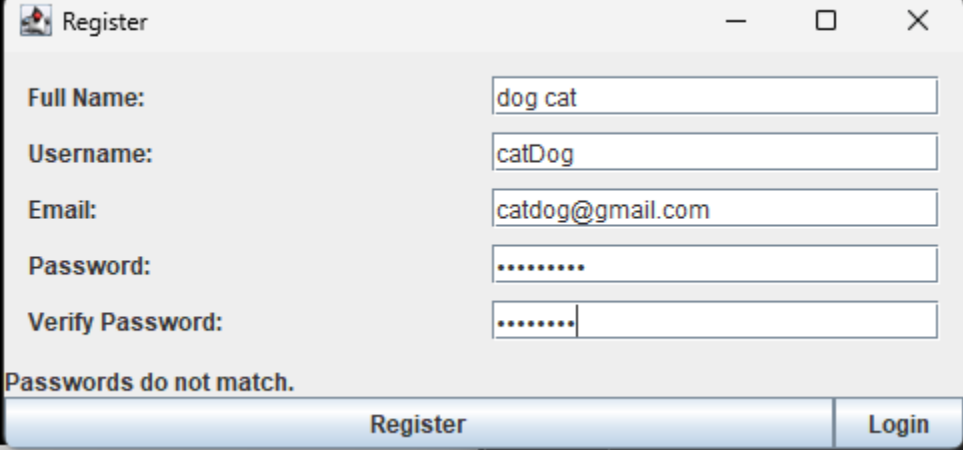
Password:

Verify Password:

Fill out your details. Or click Login to go back.

Register Login

This screenshot shows a 'Register' dialog box with a light gray background and a standard Windows-style title bar with minimize, maximize, and close buttons. The dialog contains five text input fields: 'Full Name' (containing 'dog cat'), 'Username' (containing 'catDog'), 'Email' (containing 'catdog@gmail.com'), 'Password' (containing seven dots), and 'Verify Password' (containing seven dots). Below the fields is a line of text: 'Fill out your details. Or click Login to go back.' At the bottom of the dialog are two buttons: 'Register' and 'Login'.



Register

Full Name: dog cat

Username: catDog

Email: catdog@gmail.com

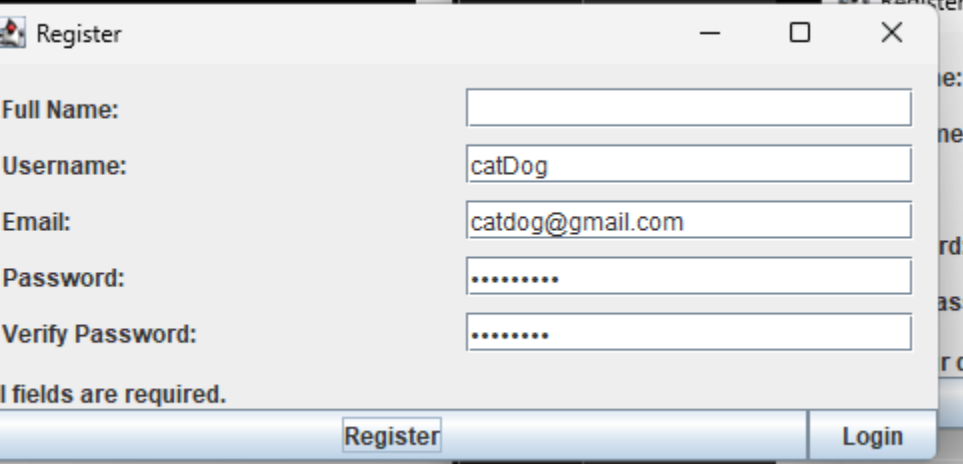
Password:

Verify Password:

Passwords do not match.

Register Login

This screenshot shows the same 'Register' dialog box as the first one, but with an error message 'Passwords do not match.' displayed below the 'Verify Password' field. The input fields contain the same data as in the first screenshot.



Register

Full Name:

Username: catDog

Email: catdog@gmail.com

Password:

Verify Password:

All fields are required.

Register Login

This screenshot shows the 'Register' dialog box with an error message 'All fields are required.' displayed below the 'Full Name' field. The 'Full Name' field is now empty, while the other fields ('Username', 'Email', 'Password', 'Verify Password') still contain their previous values.

Use main.java as single
launch point

#lang racket

```
(define (let (let (c
```

Register

Full Name: dog cat

Username: catDog

Email: catdoggmail.com

Password:

Verify Password:

Email must be in the format: example@domain.com

Register Login

;; mail
;; Runs the parser on the given filename.
;; On success, prints the AST. On any error, prints the error message as plain
(define (main filename)

Register

Full Name: dog cat

Username:

Email:

Password:

Verify Password:

Email must be in the format: example@domain.com

Register Login

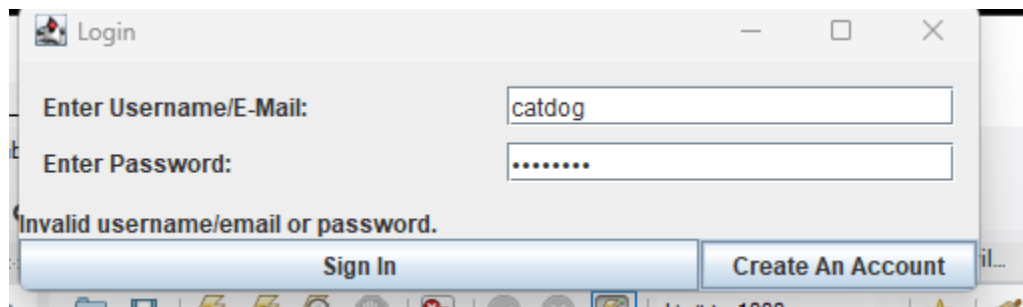
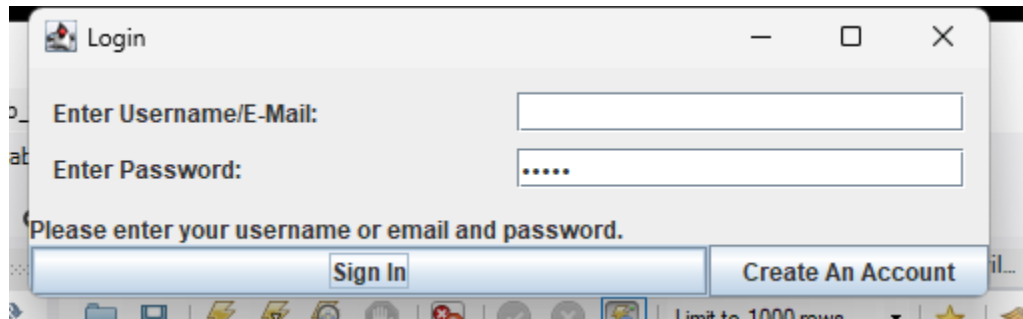
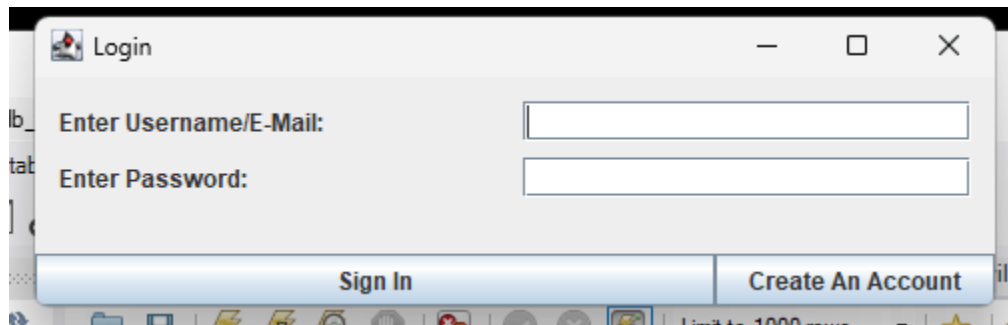
Message

Registration successful! Please log in.

OK

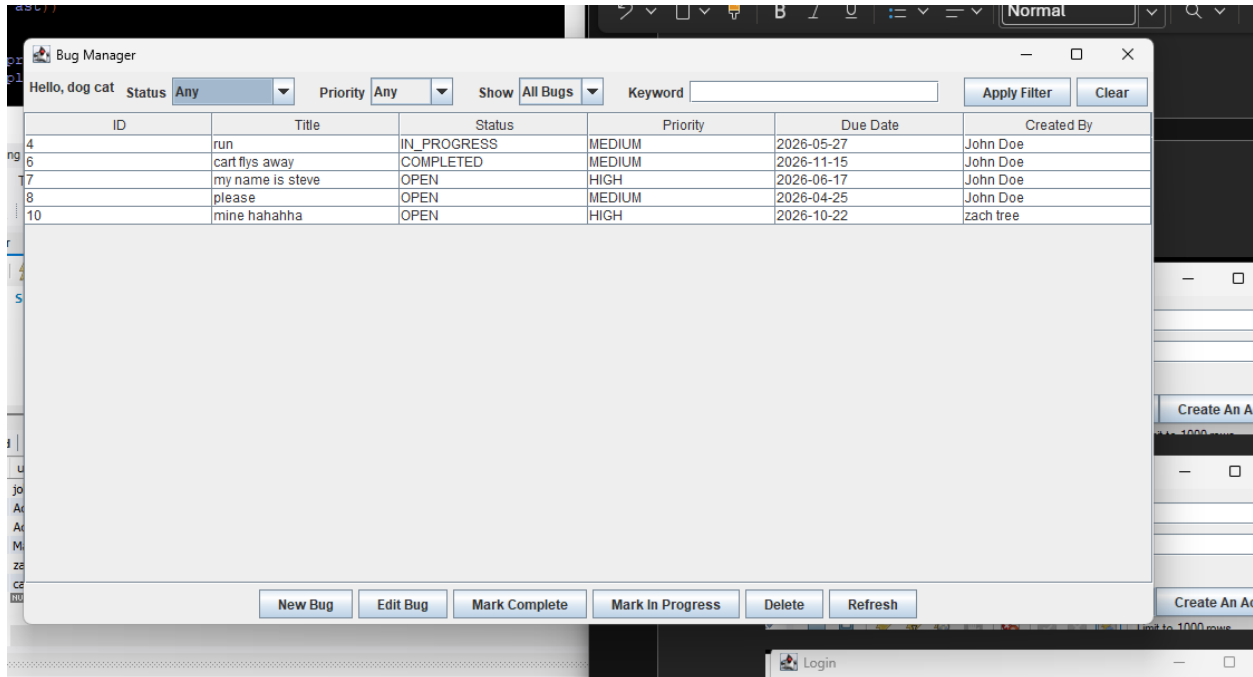
Result Grid						
Filter Rows:						
	id	username	fullname	email	password	is_ad
▶	1	johndoe	John Doe	johndoe@email.com	password123	0
	4	AdBaf	Adu Baffour	aadnxq@umkc.edu	AduRules1	0
	6	AdBaf2	Adu Baffour	aabnxq@umkc.edu	A12345678	0
	7	Mackadoodle48	Mackenzi Mason	mackenzi1renee@gmail.com	Clovercats1	0
	8	zat	zach tree	zat@umkc.edu	w5/E/k2z+f9Qu...	0
	9	catDog	dog cat	catdog@gmail.com	beDxLscFhrChR...	0
*	NULL	NULL	NULL	NULL	NULL	NULL

Use main.java as single
launch point



Use main.java as single
launch point

On login goes straight here



Requirements

1) In-Memory Backend (DAO + Service)

You must support data storage in memory using Java collections.

Required:

- DAO interface + in-memory implementation for your main entity
- Service interface + implementation that calls the DAO
- At least 1 business rule enforced in the service layer (throw `IllegalArgumentException`)

No database yet. Data resets when the app closes.

2) Swing UI: CRUD for One Main Entity

Your UI must provide a CRUD interface for **one main entity** (your choice).

Required UI components:

- a main window (JFrame)
- a `JTable` (or equivalent) showing multiple records
- buttons or menu actions for:
 - **Refresh/List All**

Use main.java as single launch point

- **New/Create**
- **Edit/Update**
- **Delete**
- a create/edit dialog (JDialog) with:
 - input fields for key attributes
 - OK / Cancel
 - edit mode should populate existing values

Required behavior:

- UI must update the table after create/update/delete
- Delete must require confirmation
- UI must display success/status messages somewhere (status label preferred)

3) MVC / Layering (Required)

Your code must show clear separation:

- UI classes must not contain collection manipulation logic
- UI must call the controller (not the DAO directly)
- Controller calls the service
- Service calls the DAO

You should be able to explain how data flows from a button click to the backend and back to the table.

4) Error Handling & Feedback

- Validation errors must show a friendly dialog message
- Attempting to edit/delete without selecting a row must show a friendly message
- The app should not crash on common user mistakes

Use main.java as single
launch point

How This Will Be Graded

Functionality (50%)

- CRUD workflow works for one entity
- UI updates correctly after operations
- In-memory backend supports required operations

UI Quality & Usability (20%)

- table view is readable and useful
- dialogs are clear and usable
- basic feedback messages are present

Architecture (MVC + layering) (20%)

- UI → Controller → Service → DAO separation is clear
- no backend logic inside UI code

Documentation (10%)

- milestone brief is complete and clear
- screenshots demonstrate required features

Please note i continued anyway

Use main.java as single
launch point

MySQL Workbench



mysqldump Version Mismatch

mysqldump.exe is version 8.0.45, but the MySQL Server to be dumped has version 9.6.0.
Because the version of mysqldump is not the same as the server, some features may not be backed up properly.
It is recommended you upgrade or downgrade your local MySQL client programs, including mysqldump, to a version equal to or newer than that of the target server.
The path to the dump tool must then be set in Preferences -> Administrator -> Path to mysqldump Tool:

Continue Anyway

Cancel

Spring 2026 java

Data Export

Advanced Options...

Object Selection

Export Progress

Export Completed

Status:

2 of 2 exported.

Log:

01:31:25 Dumping bug_tracker (bugs)
Running: mysqldump.exe --defaults-file="C:\Users\insan\AppData\Local\Temp\tmpv22f9jx8.cnf" --host=127.0.0.1 --port=3306 --default-character-set=utf8 --user=root --protocol=tcp --skip-triggers "bug_tracker" "bugs"
Warning: A partial dump from a server that has GTIDs will by default include the GTIDs of all transactions, even those that changed suppressed parts of the database. If you don't want to restore GTIDs, pass --set-gtid-purged=OFF. To make a complete dump, pass --all-databases --triggers --routines --events.
Warning: A dump from a server that has GTIDs enabled will by default include the GTIDs of all transactions, even those that were executed during its extraction and might not be represented in the dumped data. This might result in an inconsistent data dump.
In order to ensure a consistent backup of the database, pass --single-transaction or --lock-all-tables or --master-data.

01:31:25 Export of C:\Users\insan\OneDrive\Documents\dumps\Dump20260330 (1) has finished